JASKON-TECH

Engineering modern digital structures for physical design pioneers

---

DCIT 208 — Software Engineering

D4: Software Architecture & Design, Repository Engineering Baseline, and Deployment Plan

TEAM NAME	JASKON-Tech
PROJECT TITLE	ArchiVerse — A Centralized Project Tracking and Client Engagement System for Asamoah Welbeck & Associates
CLIENT	Asamoah Welbeck & Associates
DELIVERABLE	D4 — Software Architecture, Repository Engineering & Deployment Plan
TEAM REPOSITORY	github.com/Dept-of-Comp-Sci-University-of-Ghana/dcit208-client-project-2026-team-engineering-repository-seg26-44-jaskon-tech
RELEASE TAG / COMMIT	Exploring Architecture beyond Blueprints
SUBMISSION DATE	17th July, 2026

TEAM MEMBERS

Name	Student ID	GitHub Username
Kelly Kristine Donkor	22407052	KellyKristine368
Essuman Patrick	22407048	deberryy
Joseph Kwaku Wiafe	22374055	jkwiafe
Osei-Hene Owuraku Danquah	22372680	owura6789
Shanell Sarkodie	22371334	sarkodieshanell
Adwoa Nyameye Ani Asamoah	22412447	Nyameye
Asi Nyama Frimpong-Anin	22371062	nyama-aa

AI DISCLOSURE STATEMENT

AI assistance was used to structure and format this document, and to help organize the architecture description, database design, repository engineering practices, and deployment plan from work completed by the team. All architecture decisions, diagrams, API contracts, and deployment strategies reflect the team's actual work and were reviewed and verified before submission.

A. Software Architecture and Design

Purpose

The purpose of this document is to describe the overall design of the ArchiVerse Architectural Project Management System. The document translates the requirements gathered during the Software Requirements Specification (SRS) into a technical design that will guide the implementation of the system. It serves as a blueprint for developers by describing the architecture, database design, software components, interfaces, and interactions between different parts of the system.

System Overview

ArchiVerse is a web-based architectural project management platform that enables clients and architectural firms to collaborate efficiently throughout the project lifecycle. The system allows clients to register, create projects, upload project requirements, book consultations with architects, exchange messages, monitor project progress, and receive notifications. Architects and administrators can manage projects, communicate with clients, review uploaded documents, and maintain company portfolios and services.

Design Goals

The design of ArchiVerse aims to achieve the following objectives:

- Provide a simple and user-friendly interface.
- Ensure secure user authentication and role-based authorization.
- Support efficient management of architectural projects.
- Enable secure storage and management of project documents.
- Allow effective communication between clients and architects.
- Build a scalable architecture that supports future enhancements.
- Maintain high system reliability, maintainability, and performance.

Technology Stack

The system will be developed using the following technologies:

Component	Technology
Frontend	React.js
Backend	Django REST Framework
Database	PostgreSQL
Authentication	Django Authentication with JWT
Version Control	Git and GitHub
API Testing	Postman
UI Design	Figma
UML Diagrams	draw.io
Development Environment	Visual Studio Code

Assumptions

The following assumptions were made during the system design:

- Users have access to a stable internet connection.

- All users access the system using modern web browsers.
- PostgreSQL will be used as the primary database.
- Authentication will be handled securely using JWT.
- Email services will be available for password reset and notifications.
- All uploaded documents comply with supported file formats.

Constraints

The system is subject to the following constraints:

- Development will follow the Scrum methodology.
- The project will be completed within the semester timeline.
- Only authorized users may access restricted system features.
- The system must comply with basic data privacy and security principles.

Architectural Style

The ArchiVerse system follows a three-tier architecture consisting of the Presentation Layer, Application Layer, and Data Layer. This architecture separates the user interface, business logic, and data storage into independent layers, making the system easier to maintain, test, and scale.

The system also follows a client-server architecture, where the frontend communicates with the backend through RESTful APIs, and the backend interacts with the PostgreSQL database to process and store data.

Architecture Components

Presentation Layer (Frontend)

The Presentation Layer is responsible for interacting with users. It provides the graphical user interface where clients, architects, and administrators can perform system operations.

The frontend will be developed using React.js and will communicate with the backend using HTTP requests to REST APIs.

Functions include:

- User Registration
- User Login
- Dashboard
- Project Management
- Consultation Booking
- Portfolio Viewing
- Messaging
- Notifications
- Contact Forms

Application Layer (Backend)

The Application Layer contains the business logic of the system. It processes requests received from the frontend, validates user input, performs authentication and authorization, communicates with the database, and returns appropriate responses.

The backend will be developed using Django REST Framework (DRF). Major responsibilities include:

- User Authentication

- Role Management
- Project Management
- Consultation Scheduling
- Blueprint Management
- Messaging Services
- Notification Services
- Report Generation

Data Layer

The Data Layer stores and manages all application data.

The system will use PostgreSQL as its primary relational database. Uploaded files such as architectural blueprints will be stored in the media storage directory while their metadata is stored in the database.

The database stores information about:

- Users
- Projects
- Blueprints
- Consultations
- Messages
- Notifications
- Portfolio
- Services
- Reports

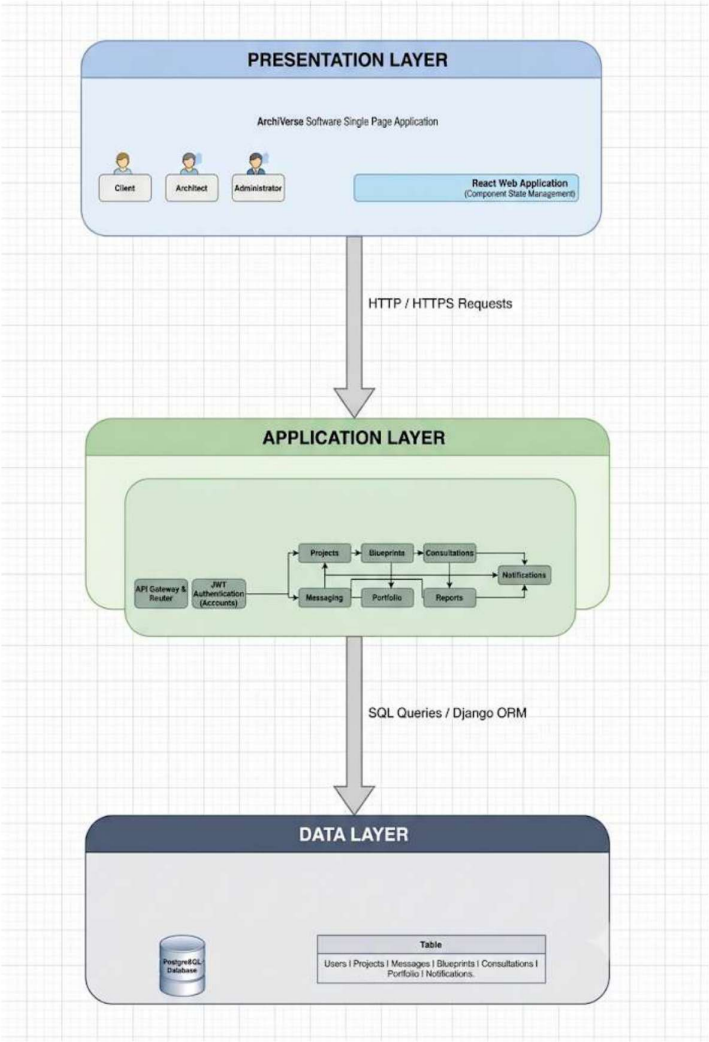


Figure A.1 — Three-Tier Architecture: Presentation, Application, and Data Layers

Communication Flow

The communication flow within the system is as follows:

1. The user interacts with the React frontend through a web browser.
2. The frontend sends an HTTP request to the Django REST API.
3. Django validates the request and executes the required business logic.
4. The backend communicates with the PostgreSQL database to retrieve or update data.
5. The database returns the requested information.
6. Django formats the response as JSON.
7. React receives the response and updates the user interface.

Security Architecture

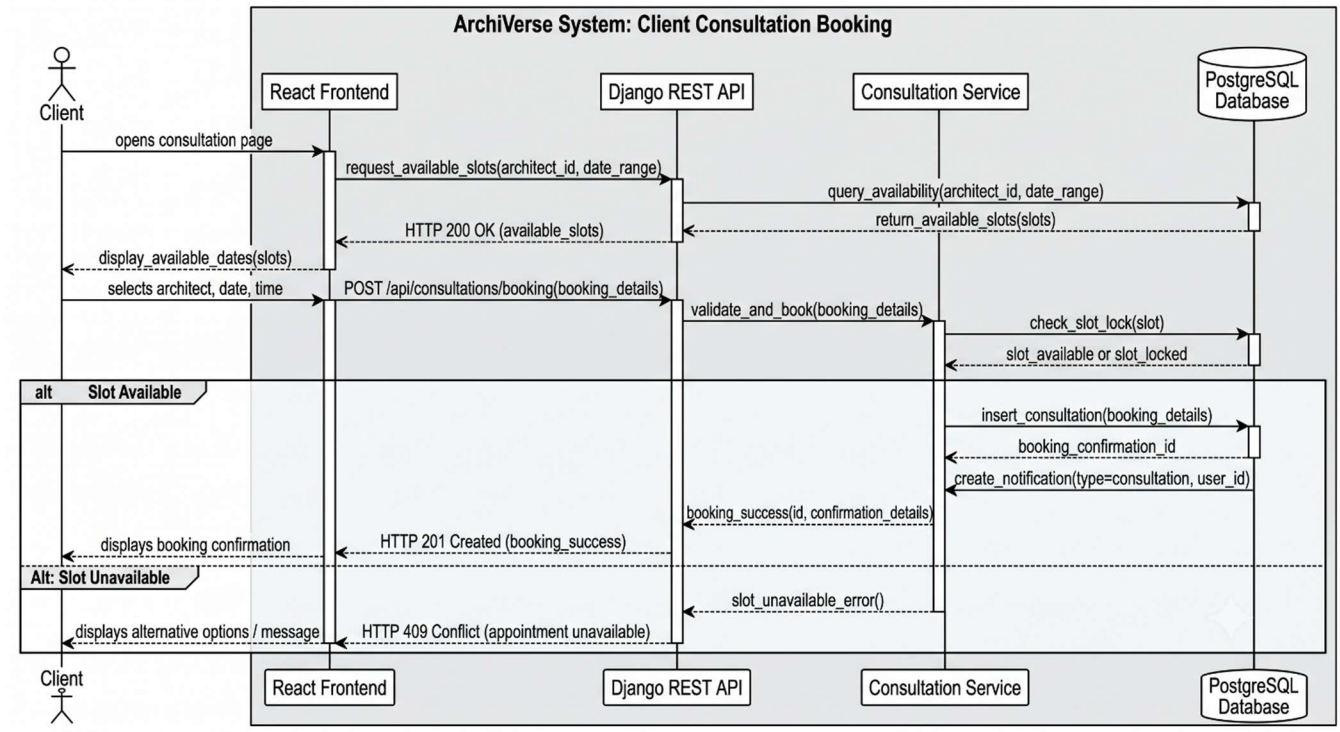
The system implements several security mechanisms to protect user data and system resources. These include:

- JWT Authentication
- Password Hashing
- Role-Based Access Control (RBAC)

- Input Validation
- CSRF Protection
- Secure API Endpoints
- HTTPS Deployment
- Permission-based Authorization

UML Class Diagram

The UML Class Diagram presents the main software classes within the system, their attributes, and the relationships between them. It serves as the blueprint for implementing the backend using Django models and supports the overall object-oriented design of the application.



Database Overview

The ArchiVerse system uses a PostgreSQL relational database to store and manage application data. The database is designed to ensure data integrity, reduce redundancy through normalization, and support efficient retrieval of information. Relationships between entities are implemented using primary keys and foreign keys to maintain consistency across the system.

Main Data Entities

The major entities in the system are listed below.

Entity	Description
User	Stores information about clients, architects, and administrators.
Project	Stores architectural project details created by clients.
Blueprint	Stores uploaded architectural drawings and project documents.
Consultation	Stores consultation bookings between clients and architects.
Message	Stores communication between users.
Notification	Stores system notifications sent to users.

Portfolio	Stores completed projects displayed by the company.
Service	Stores architectural services offered by the company.
Contact Message	Stores enquiries submitted through the Contact Us page.

Entity Relationships

The relationships between the entities are as follows:

- One User can create many Projects.
- One User (Architect) can manage many Projects.
- One Project can have many Blueprints.
- One Client can book many Consultations.
- One Architect can receive many Consultations.
- One User can send many Messages.
- One User can receive many Messages.
- One User can receive many Notifications.
- One Portfolio can contain multiple images or completed project records.
- One Service can be viewed by many users.

Primary Keys

Entity	Primary Key
User	user_id
Project	project_id
Blueprint	blueprint_id
Consultation	consultation_id
Message	message_id
Notification	notification_id
Portfolio	portfolio_id
Service	service_id
Contact Message	contact_id

Data Integrity Constraints

To ensure the reliability and consistency of data, the following constraints will be applied:

- Every user must have a unique email address.
- Passwords will be stored in encrypted form.
- A project cannot exist without an associated client.
- A blueprint must belong to an existing project.
- A consultation must reference both a client and an architect.
- Foreign key relationships will enforce referential integrity.
- Required fields must not contain null values unless explicitly allowed.

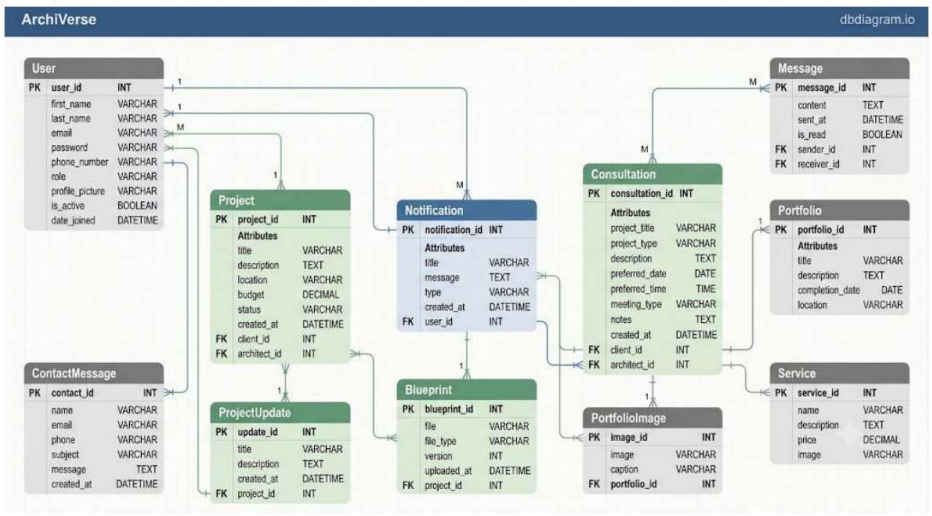


Figure A.2 — Entity Relationship Diagram (ERD)

API / Interface Contract

The system exposes RESTful APIs through Django REST Framework.

Module	Endpoint	Method	Purpose
Authentication	/api/auth/register/	POST	Register a new user
Authentication	/api/auth/login/	POST	Authenticate user
Authentication	/api/auth/logout/	POST	Logout user
Projects	/api/projects/	GET	View projects
Projects	/api/projects/	POST	Create project
Projects	/api/projects/{id}/	PUT	Update project
Projects	/api/projects/{id}/	DELETE	Delete project
Consultation	/api/consultations/	POST	Book consultation
Consultation	/api/consultations/	GET	View consultations
Blueprint	/api/blueprints/	POST	Upload blueprint
Portfolio	/api/portfolio/	GET	View architect portfolio
Portfolio	/api/portfolio/	POST	Add portfolio item
Messages	/api/messages/	POST	Send message
Notifications	/api/notifications/	GET	Retrieve notifications
Contact	/api/contact/	POST	Send contact message

All endpoints return JSON responses and require authentication where appropriate.

Architecture Decision Records (ADRs)

ADR-001: Frontend Framework

Decision

- React.js will be used for the frontend.

Reason

- React provides reusable components, efficient rendering, and integrates well with REST APIs.

Impact

- The frontend can be developed independently from the backend, making future maintenance easier.

ADR-002: Backend Framework

Decision

- Django REST Framework will be used for backend development.

Reason

- DRF provides secure authentication, rapid development, ORM support, and REST API functionality.

Impact

- The backend becomes scalable, secure, and easier to extend.

ADR-003: Database Selection

Decision

- PostgreSQL will be used as the primary database.

Reason

- PostgreSQL offers strong relational support, data integrity, indexing, and excellent compatibility with Django.

Impact

- The system can efficiently manage relationships between users, projects, consultations, messages, and blueprints while supporting future scalability.

A.6 Security and Privacy Design

The following security measures will be implemented within ArchiVerse:

- Passwords will be securely hashed before storage.
- JWT authentication will protect secured API endpoints.
- Role-Based Access Control (RBAC) will restrict access based on user roles (Client, Architect, Administrator).
- Input validation will prevent invalid or malicious data from entering the system.
- HTTPS will encrypt communication between the client and server.
- Uploaded blueprint files and portfolio images will be validated before storage.
- Sensitive user information will only be accessible to authorized users.
- Database backups will be performed regularly to prevent data loss.

These measures ensure the confidentiality, integrity, and availability of system data.

B. Repository Engineering Baseline

Makefile

The project includes a root Makefile to simplify common development tasks. The Makefile provides standardized commands that allow all team members to set up, run, test, and maintain the project consistently.

The Makefile contains the following commands:

Command	Purpose
make setup	Creates the virtual environment, installs dependencies, and prepares the development environment.
make run	Starts the Django backend development server.
make lint	Runs code quality checks using Ruff or Flake8.
make test	Executes automated unit tests.
make migrate	Applies database migrations.
make makemigrations	Generates new database migration files.

GitHub Actions Continuous Integration (CI)

A GitHub Actions workflow has been configured to automatically verify code quality whenever a Pull Request is opened against the main branch.

The workflow performs the following steps:

- Checks out the repository.
- Sets up Python.
- Installs project dependencies.
- Runs linting checks.
- Executes automated tests.
- Reports whether the Pull Request passes or fails.

Automated CI helps detect coding errors early and prevents unstable code from being merged into the main branch.

Pull Request Template

A Pull Request template has been created to ensure that every contribution follows the same review process.

Each Pull Request requires contributors to provide:

- A summary of the implemented feature.
- The related GitHub Issue number.
- Screenshots (for frontend changes).
- Test evidence.
- AI assistance disclosure.
- Confirmation that the code has been tested locally.

This template improves code review quality and project traceability.

GitHub Issue Templates

The repository contains Issue Templates for different categories of work. The templates include:

Template	Purpose
Feature Request	Tracks new system functionality.
Bug Report	Reports software defects.

Documentation	Tracks documentation improvements.

Using standardized issue templates ensures that every issue contains sufficient information before development begins.

CODEOWNERS

A CODEOWNERS file has been configured to protect important project files.

The following files require review before modification:

- Makefile
- GitHub Actions workflows
- Requirements files
- Architecture documentation
- Deployment configuration

This ensures that critical project files cannot be modified without approval from designated reviewers.

Branch Protection Strategy

The project follows a GitHub Flow workflow.

The main branch is protected by the following rules:

- Direct commits to the main branch are not permitted.
- All changes must be submitted through Pull Requests.
- Pull Requests must pass Continuous Integration checks before merging.
- Code reviews are required before approval.
- Resolved conversations are required before merging.

This strategy helps maintain a stable and reliable codebase throughout development.

Repository Quality Assurance

The repository engineering practices adopted for ArchiVerse include:

- GitHub Projects for Sprint Planning.
- GitHub Issues for backlog management.
- Pull Requests for code review.
- GitHub Actions for automated testing.
- Branch protection rules.
- Standardized commit messages.
- Documentation stored within the repository.
- Version control using Git.

These practices promote collaboration, maintainability, and software quality throughout the project lifecycle.

C. Deployment and Environment Plan

Chosen Deployment Platform

The ArchiVerse system will be deployed using DigitalOcean App Platform.

DigitalOcean App Platform was selected because it provides automated deployment directly from GitHub repositories, supports both Django and React applications, offers managed PostgreSQL databases, automatic HTTPS, scalability, and continuous deployment capabilities.

The platform also simplifies application management by reducing server configuration complexity, allowing the development team to focus on implementing system features rather than infrastructure management.

C.2 Deployment Architecture

The deployment architecture consists of the following components:

- Frontend: React.js application hosted on DigitalOcean App Platform.
- Backend: Django REST Framework application hosted on DigitalOcean App Platform.
- Database: Managed PostgreSQL database provided by DigitalOcean.
- Media Storage: DigitalOcean Spaces (or local media storage during development).
- Version Control: GitHub Repository.
- Continuous Integration: GitHub Actions.
- Continuous Deployment: Automatic deployment from the GitHub main branch.

This deployment architecture ensures that the frontend, backend, and database remain independent while communicating securely through REST APIs.

Environment Variables

Sensitive information will not be stored directly in the source code. Instead, the following environment variables will be configured within DigitalOcean App Platform.

Variable	Purpose
SECRET_KEY	Django secret key
DEBUG	Enable or disable debug mode
DATABASE_URL	PostgreSQL database connection string
ALLOWED_HOSTS	List of allowed domains
CORS_ALLOWED_ORIGINS	Frontend URL allowed to access the API
EMAIL_HOST	SMTP server
EMAIL_PORT	SMTP port
EMAIL_HOST_USER	Email username
EMAIL_HOST_PASSWORD	Email password
DEFAULT_FROM_EMAIL	Sender email address
CLOUDINARY_URL (or DigitalOcean Spaces credentials)	Media file storage configuration

Local Development Setup

A new developer can set up the project by following these steps:

1. Clone the GitHub repository.
2. Navigate to the backend directory.
3. Create and activate a Python virtual environment.
4. Install backend dependencies using: pip install -r requirements.txt.
5. Configure the .env file with the required environment variables.
6. Apply database migrations: python manage.py migrate.
7. Navigate to the frontend directory.
8. Install frontend dependencies: npm install.
9. Start the React development server: npm run dev.

The application will now be available locally for development and testing.

Staging and Final Release Plan

The project will use two deployment environments:

Development Environment

Used by developers to implement and test new features locally.

Staging Environment

Used to test completed features before deployment to production. The staging environment mirrors the production configuration and allows the team to identify issues before release.

Production Environment

The production environment is hosted on DigitalOcean App Platform and is accessible to end users. Only reviewed and tested features will be deployed to production.

Backup and Recovery Plan

To prevent data loss, the following backup strategy will be adopted:

- Regular backups of the PostgreSQL database.
- Version control of all source code using GitHub.
- Cloud storage for uploaded blueprint files and portfolio images.
- Rollback to previous GitHub releases if deployment issues occur.
- Restoration of database backups in the event of system failure.

This backup strategy helps ensure business continuity and protects client information.

Rollback Strategy

If a deployment introduces critical issues, the following recovery process will be followed:

1. Suspend new deployments.
2. Roll back to the most recent stable GitHub release.
3. Restore the latest verified database backup if necessary.

4. Investigate and resolve the issue.
5. Redeploy the corrected version after successful testing.

This approach minimizes downtime and ensures the system remains reliable.